实验 十二　封装与继承

一、实验目的：

1.掌握类的继承的用法

2.掌握类的继承中 this、super 关键字的用法

二、实验内容：

1．分析程序的结果，并上机验证。

```
class    Sa{
   int x,y;
   public void setXy(int x,int y) {
      this.x=x;
        this.y=y;
   }
   public int result()    {
return x+y;  }
}
class Sb extends Sa{
   int x;
   public int result(){
         super.x=5;
         x=super.result()/x;
         return x;
} }
class Sc {
public static void main(String aa[]){         Sa s=new Sa();
         s.setXy(5,10);
   System.out.println("sa result:"+s.result());
         Sb s2=new Sb();
         s2.x=3;
         s2.y=10;
         System.out.println("sb result:"+s2.result());
         }}
```

输出结果为：

2.阅读程序，分析程序的输出结果。

```
class A {
    protected void print(String s){
        System.out.println(s);
    }
    A(){print("A()");}
    public void f() {print("A:f()");}
}

class B extends A{
    B(){print("B()");}
    public void f() {print("B:f()");}
    public static void main(String arg[]) {
        B b = new B(); b.f();
    }
}
```

输出结果为：

3.定义一个矩形类，它包含两个数据成员：长和宽，以及包含用于求面积的成员方法。再定义矩形类的子类—长方体类，增加一个数据成员高以及用来求长方体体积的成员方法。在主方法中用其求长方形的面积和长方体的体积。

分析：这个题目我们在前面学习类与对象时，采用了面向对象的思想这样实现的：

第一步：封装长方形的类

```
class Rect {
    double l,w;
    Rect(double l,double w) {
        this.l=l;
        this.w=w;
    }
    double getL() {
        return l;
    }
    void setL(double l) {
        this.l=l;
    }
    double getArea() {
        return l*w;
    }
}
```

第二步：封装长方体类

```
class Cuboid {//长方体类：面（长方形）+h
    Rect r;
    double h;
    Cuboid(Rect r,double h) {
        this.r=r;
        this.h=h;
    }
    double getV() { //体积=面积*h
     return r.getArea()*h;
    }

}
```

第三步：测试类

```
public class TestRect {
    public static void main (String[] args) {
        Rect r = new Rect(3,4);//先构造长方形
        Cuboid c = new Cuboid(r,8);//在上面长方形的基础上构造长方体
        System.out.println("面积是: "+c.getV());
    }
}
```

请用刚学习的继承来实现长方体类的定义。

源程序：

```
public class 长方体类 {

	public static void main(String[] args) {
		// TODO 自动生成的方法存根
		changfangti a=new changfangti(3,4,5);
		System.out.println(a.mianji());
		System.out.println(a.tiji());
	}

}
class juxing1{
	double a,b;
	juxing1(double a,double b){
		this.a=a;
		this.b=b;
	}
	double mianji() {
		return a*b;
	}
}
class changfangti extends juxing1{
	double gao;
	double mianji;
	changfangti(double a,double b,double gao){
		super(a,b);
		this.gao=gao;
	}
	double tiji(){
		return mianji()*gao;
	}
}
```